GANs – Part 2: Supervised regression with an extra GAN loss

Author: Sebastian Bujwid bujwid@kth.se

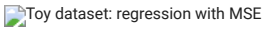
Refer to ./README.md for general comments.

Objectives

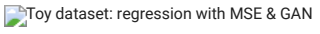
In this part of the practical, we will demonstrate that GAN training can be utilized not only for unsupervised generation of realistic samples but also, for example, for helping supervised training.

Introduction

In supervised learning, specifying the loss function in some cases might actually be quite challenging. Standard loss functions, like cross-entropy or mean squared error (MSE), typically assume that the output closer to the ground-truth is better. Even though it might seem like a reasonable assumption, it might not work great in the presence of higher uncertainties. That could be, for example, when multiple answers seem possible but the model has no way of telling which one of them is true - then the best thing it can do (with respect to the loss function) is to average the answers that seem possible. Is the average of two possible outputs a good answer, though? Not necessarily! Consider the figure below with some example toy regression problem which is used in this assignment:



MSE loss:
the region in the center (connecting the two regions of data) might minimize MSE but the data in that regions is very unlikely to occur.



MS
the model learns to (almost) avoid the region in the center w

Which of the models is better? That really depends on the problem you're trying to solve and the properties of the solution you care about.

A similar example to illustrate potential benefit of using GAN for supervised problem could be for example super-resolution: 

Super-resolution: (Left) Optimized for MSE; (Right) Using GAN

Source: SRGAN

The image produced by the model using MSE loss is blurry, while the one generated by the model trained with GAN loss is much sharper.

In some sense, one could think of the discriminator as a loss function that is learned and which represents whether the model's output is similar to the real data samples. As it would be difficult to specify such a loss function manually, the concept of GAN training (or using a discriminator) can be used for many different problems and in many different settings, even supervised training. In this part of the practical, we will use the GAN training on the supervised regression problem on a toy dataset.

Note, however, that in the case of our specific regression example, uncertainty estimation methods could be more suitable for many purposes.

Scope

- Train a supervised regression model with a mean squared error (MSE) loss
- Add a GAN loss to the supervised regression problem & train the model

---

```
import numpy as np
import sklearn.datasets
import sklearn.model_selection
import matplotlib.pyplot as plt
import seaborn as sns
import itertools


seed = 42
```

Toy dataset

```
def dataset_noisy_lines(n_samples, seed):

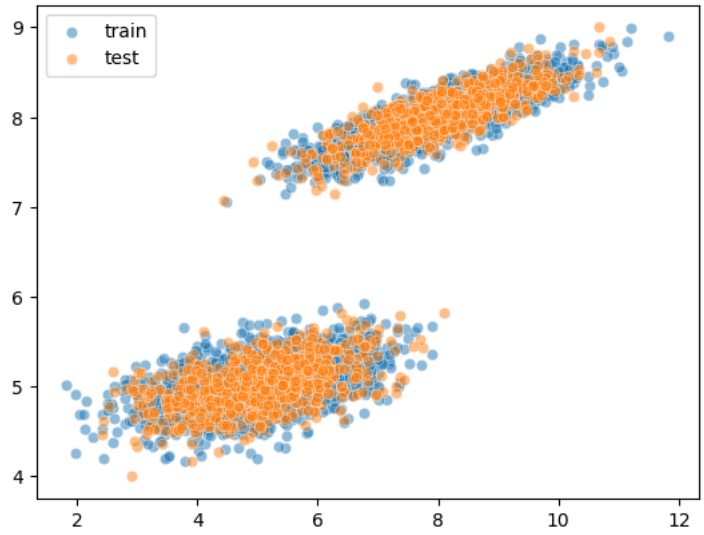
    def make_line(n_samples, seed):
        x, y = sklearn.datasets.make_regression(
            n_samples=n_samples // 2,
            n_features=1,
            n_informative=1,
            bias=0.5,
            effective_rank=None,
            tail_strength=0.,
            noise=5.0,
            shuffle=True,
            coef=False,
            random_state=seed)
```

```
        return np.concatenate([x, np.expand_dims(y / np.max(np.abs(y)), -1)], axis=-1)

    line1 = make_line(n_samples // 2, seed=seed + 1)
    line2 = make_line(n_samples // 2, seed=seed + 2) + np.array([3., 3])
    data = np.concatenate([line1, line2], axis=0) + 5.
    x, y = data[:, [0]], data[:, [1]]
    x_train, x_test, y_train, y_test = sklearn.model_selection.train_test_split(
        x, y,
        test_size=0.3,
        random_state=seed,
        shuffle=True,
    )
    return (x_train, y_train), (x_test, y_test)


data_train, data_test = dataset_noisy_lines(n_samples=10000, seed=seed)
print(data_train[0].shape)
print(data_train[1].shape)
sns.scatterplot(alpha=0.5, x=data_train[0][:, 0], y=data_train[1][:, 0], label='train')
sns.scatterplot(alpha=0.5, x=data_test[0][:, 0], y=data_test[1][:, 0], label='test')
plt.show()
```

```
(3500, 1)
(3500, 1)
```



Implementation

```
import jax
import jax.numpy as jnp
from jax.example_libraries  import stax
from jax.example_libraries  import optimizers
from jax import jit
from jax import grad



#from utils import plot_losses

import matplotlib.pyplot as plt


def plot_losses(losses_dict):
    for key, vals in losses_dict.items():
        plt.plot(vals, label=key)
    plt.legend()
    plt.show()
```

Mean squared error (MSE) loss

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

```
def mse_loss(targets, predictions):
    assert targets.ndim == predictions.ndim

    return jnp.mean(jnp.square(predictions - targets), axis=-1)
```

```
def iterable_dataset(key, data, batch_size):
    data_x, data_y = data
    assert len(data_x) == len(data_y)
    n_samples = len(data_x)
    n_batches = n_samples // batch_size
    x_shuffled, y_shuffled = jnp.split(
        jax.random.permutation(
            key, jnp.concatenate([data_x, data_y], axis=-1)
        ),
        2, axis=-1
    )
    dataset = [
        (
            x_shuffled[i * batch_size: (i + 1) * batch_size],
            y_shuffled[i * batch_size: (i + 1) * batch_size],
        ) for i in range(n_batches)
    ]
    return dataset


def show_predictions(model_params, model_apply, data):
    x, y = data
    pred_y = model_apply(model_params, x)
    sns.scatterplot(alpha=0.5, x=x[:, 0], y=y[:, 0], label='real_data')
    sns.scatterplot(alpha=0.5, x=x[:, 0], y=pred_y[:, 0], label='predicted')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.show()


pip install omegaconf
```

```
Collecting omegaconf
  Downloading omegaconf-2.3.0-py3-none-any.whl (79 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.5/79.5 kB 1.5 MB/s eta 0:00:00
Collecting antlr4-python3-runtime==4.9.* (from omegaconf)
  Downloading antlr4-python3-runtime-4.9.3.tar.gz (117 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.0/117.0 kB 6.3 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: PyYAML>=5.1.0 in /usr/local/lib/python3.10/dist-packages (from omegaconf) (6.0.1)
Building wheels for collected packages: antlr4-python3-runtime
  Building wheel for antlr4-python3-runtime (setup.py) ... done
  Created wheel for antlr4-python3-runtime: filename=antlr4_python3_runtime-4.9.3-py3-none-any.whl size=144554 sha256=38791511da0608
  Stored in directory: /root/.cache/pip/wheels/12/93/dd/1f6a127edc45659556564c5730f6d4e300888f4bca2d4c5a88
Successfully built antlr4-python3-runtime
Installing collected packages: antlr4-python3-runtime, omegaconf
Successfully installed antlr4-python3-runtime-4.9.3 omegaconf-2.3.0
```

```
from omegaconf import OmegaConf


#from omegaconf import OmegaConf

regression_hparams = OmegaConf.create({
    'epochs': 10,
    'batch_size': 512,
    'lr': 0.0001,
    'model': {
        'n_layers': 5,
        'hidden_dim': 512,
    },
})


def create_mlp(n_hid_layers, hid_dim, output_dim):
    layers = list(itertools.chain.from_iterable(
        [(stax.Dense(hid_dim), stax.Relu) for _ in range(n_hid_layers)]
    )) + [stax.Dense(output_dim)]
    mpl_init, mpl_apply = stax.serial(*layers)
    return mpl_init, mpl_apply
```

Regression model trained with MSE

First, we will train a regression model with just the MSE loss.

```
def create_and_train_regression_model(hparams, data, test_data):

    x_dim, y_dim = 1, 1

    # Create model
    model_init, model_apply = create_mlp(
        n_hid_layers=hparams.model.n_layers,
        hid_dim=hparams.model.hidden_dim,
        output_dim=y_dim
    )

    # Initialize
    key = jax.random.PRNGKey(seed)
    key, key_model = jax.random.split(key, 2)

    x_shape = (-1, x_dim)
    _, model_params = model_init(key_model, x_shape)

    ## Initialize: optimizer
    model_opt_init, model_opt_update, model_get_params = optimizers.adam(
        step_size=hparams.lr
    )
    model_opt_state = model_opt_init(model_params)

    # Training functions
    def loss_fn(model_params, x, y):
        pred_y = model_apply(model_params, x)
        loss = jnp.mean(mse_loss(y, pred_y), axis=0)
        return loss

    @jit
    def train_step(step, model_opt_state, x, y):
        model_params = model_get_params(model_opt_state)

        loss, model_grads = jax.value_and_grad(loss_fn)(model_params, x, y)

        model_opt_state = model_opt_update(step, model_grads, model_opt_state)
        return loss, model_opt_state

    loss_history = []

    total_step = 0
    for epoch in range(hparams.epochs):
        key, key_data = jax.random.split(key)

        for batch_x, batch_y in iterable_dataset(key_data, data, batch_size=hparams.batch_size):

            loss, model_opt_state = train_step(
                step=total_step,
                model_opt_state=model_opt_state,
                x=batch_x,
                y=batch_y,
            )

            loss_history.append(loss)

            total_step += 1

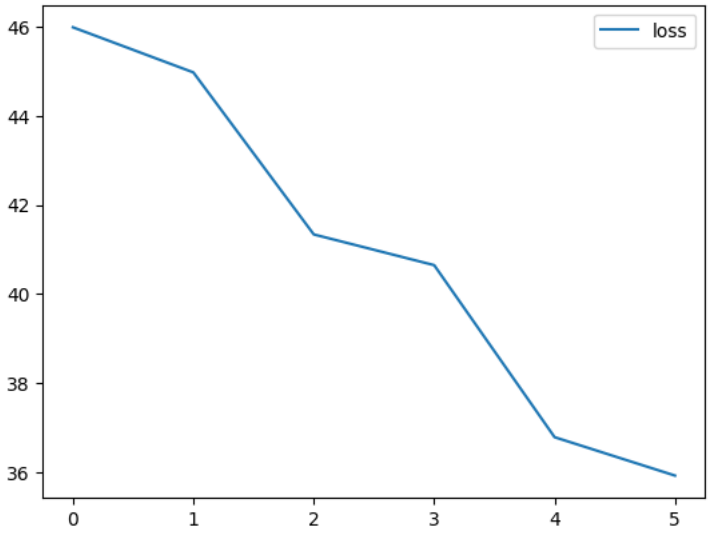
        if epoch == 0 or (epoch < 100 and epoch % 10 == 9) or epoch % 100 == 99:
            print('-' * 30, 'epoch', epoch, '-' * 30)
            plot_losses({'loss': loss_history})
            show_predictions(model_get_params(model_opt_state), model_apply, test_data)
```

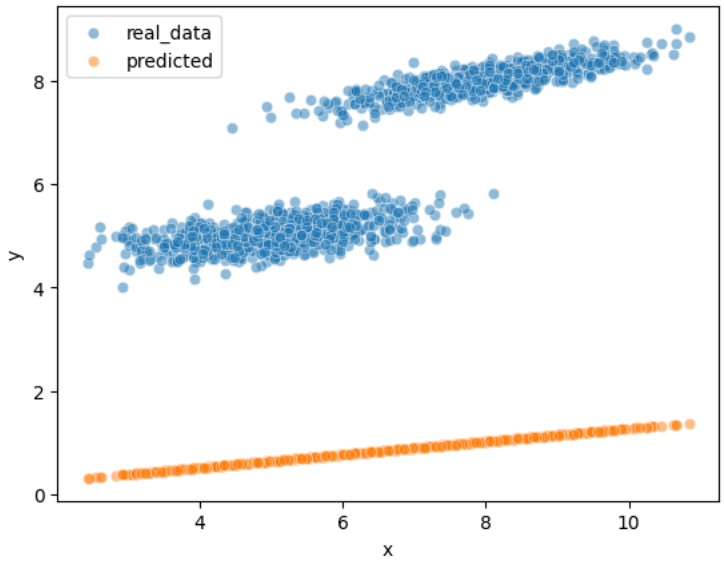
(Task) Train the regression model with MSE loss

The implementation is ready and you should be able just to run the training!

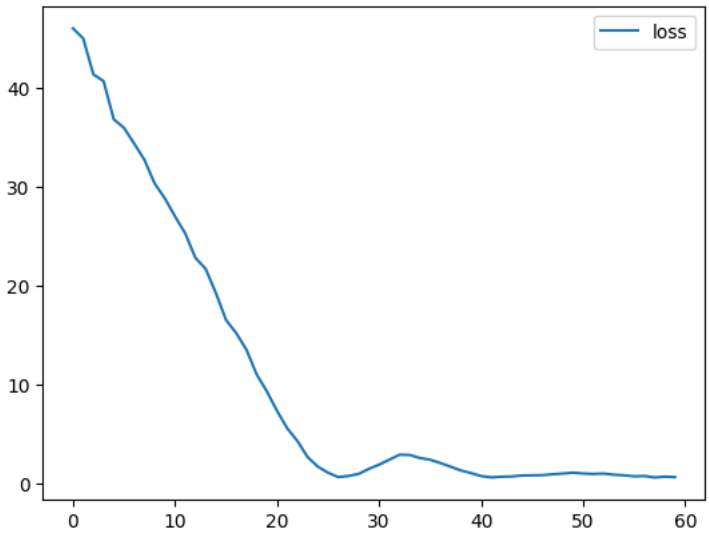
```
create_and_train_regression_model(regression_hparams, data_train, test_data=data_test)
```

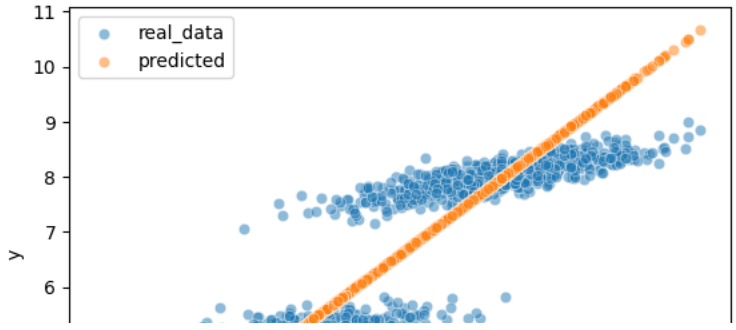
```
------------------------------ epoch 0 ------------------------------
```






```
------------------------------ epoch 9 ------------------------------
```





As you hopefully able to observe, the model trained with MSE is making predictions in the center of the figure where any samples are unlikely to happen.

Now, let's see how will the predictions change if the model is trained with an additional GAN objective.

(Task) Regression model trained with MSE + discriminator (GAN loss)

Task: Implement a regression model similar to the one before but now with an extra discriminator model. The discriminator should be trained on real vs. predicted `y` values! In such setting, we consider the regression model as a generator, which should be trained with the MSE + GAN loss. (added, possibly with a weighting factor of `hparams.gan_loss_weight`)

Note: no input noise is needed for the generator/regressor (although it is possible to use it)

```
# You can copy/re-use the functions from Part 1

#from google.colab import drive
#drive.mount('/content/drive')
#from importlib.machinery import SourceFileLoader
#module = SourceFileLoader("gan_part1_solu", "dd2412-gan-and-adv-domain-adaptation-main/gan_part1_solu.py Notebooks/gan_part1_solu.py").l


#from gan_part1_solu import sigmoid_cross_entropy, generator_loss, discriminator_loss


def sigmoid_cross_entropy(*, targets, logits):
    assert targets.shape == logits.shape
    sigmoid_logits = jax.nn.sigmoid(logits)
    loss = - (targets * jnp.log(sigmoid_logits) + (1 - targets) * jnp.log(1 - sigmoid_logits))
    assert loss.shape == logits.shape, 'cross-entropy loss is not expected here to be averaged over samples'
    return loss

def discriminator_loss(*, real_logits, fake_logits):
    real_targets = jnp.ones_like(real_logits)  # Real targets are all 1
    fake_targets = jnp.zeros_like(fake_logits)  # Fake targets are all 0
    real_loss = jnp.mean(sigmoid_cross_entropy(targets=real_targets, logits=real_logits))
    fake_loss = jnp.mean(sigmoid_cross_entropy(targets=fake_targets, logits=fake_logits))
    loss = real_loss + fake_loss
    assert loss.shape == (), 'discriminator loss is expected here to be averaged over samples'
    return loss

def generator_loss(discriminator_fake_logits):
    loss = jnp.mean(sigmoid_cross_entropy(targets=jnp.ones_like(discriminator_fake_logits), logits=discriminator_fake_logits))
    assert loss.shape == (), 'discriminator loss is expected here to be averaged over samples'
    return loss


def create_and_train_regression_model_with_discriminator(hparams, data, test_data):
    x_dim, y_dim = 1, 1

    # Create model
    model_init, model_apply = create_mlp(
        n_hid_layers=hparams.model.n_layers,
        hid_dim=hparams.model.hidden_dim,
        output_dim=y_dim
    )
    discriminator_init, discriminator_apply = create_mlp(
        n_hid_layers=hparams.discriminator.n_layers,
        hid_dim=hparams.discriminator.hidden_dim,
        output_dim=1,
    )

    # Initialize
    key = jax.random.PRNGKey(seed)
    key, key_model, key_dis = jax.random.split(key, 3)

    x_shape = (-1, x_dim)
    y_shape = (-1, y_dim)
    _, model_params = model_init(key_model, x_shape)
    _, dis_params = discriminator_init(key_dis, y_shape)

    ## Initialize: optimizer
    model_opt_init, model_opt_update, model_get_params = optimizers.adam(
        step_size=hparams.lr, b1=0.5
    )
    dis_opt_init, dis_opt_update, dis_get_params = optimizers.adam(
        step_size=hparams.dis_lr, b1=hparams.dis_beta1
    )
    model_opt_state = model_opt_init(model_params)
    dis_opt_state = dis_opt_init(dis_params)
```

```
    def compute_gradients(model_params, dis_params, x, y, hparams):
        # Forward pass for the generator (model)
        pred_y = model_apply(model_params, x)

        # Calculate MSE loss for the generator
        model_loss = jnp.mean(mse_loss(y, pred_y), axis=0)

        # Forward pass for the discriminator
        real_dis = discriminator_apply(dis_params, y)
        fake_dis = discriminator_apply(dis_params, pred_y)

        # Calculate GAN loss for the discriminator
        dis_loss = discriminator_loss(real_logits=real_dis, fake_logits=fake_dis)

        # Calculate the total generator loss (MSE + GAN loss)
        generator_loss = model_loss + hparams.gan_loss_weight * dis_loss

        # Calculate gradients
        model_grads = jax.grad(generator_loss)(model_params)
        dis_grads = jax.grad(dis_loss)(dis_params)

        return model_grads, dis_grads

    @jit
    def train_step(step, model_opt_state, dis_opt_state, x, y):
        model_params = model_get_params(model_opt_state)
        dis_params = dis_get_params(dis_opt_state)

        # Compute gradients
        model_grads, dis_grads = compute_gradients(model_params, dis_params, x, y, hparams)

        # Update the generator and discriminator
        model_opt_state = model_opt_update(step, model_grads, model_opt_state)
        dis_opt_state = dis_opt_update(step, dis_grads, dis_opt_state)

        return (model_loss, dis_loss), (model_opt_state, dis_opt_state)

    loss_history = {'model': [], 'dis': []}

    total_step = 0
    for epoch in range(hparams.epochs):
        key, key_data = jax.random.split(key)

        for batch_x, batch_y in iterable_dataset(key_data, data, batch_size=hparams.batch_size):

            (model_loss, dis_loss), (model_opt_state, dis_opt_state) = train_step(
                step=total_step,
                model_opt_state=model_opt_state,
                dis_opt_state=dis_opt_state,
                x=batch_x,
                y=batch_y,
            )

            loss_history['model'].append(model_loss)
            loss_history['dis'].append(dis_loss)

            total_step += 1

        if epoch == 0 or (epoch < 100 and epoch % 10 == 9) or epoch % 100 == 99:
            print('-' * 30, 'epoch', epoch, '-' * 30)
            plot_losses(loss_history)
            show_predictions(model_get_params(model_opt_state), model_apply, test_data)
```

(Task) Train the regression model with MSE + GAN loss

```
regression_with_discriminator_hparams = OmegaConf.create({
    'epochs': 20,
    'batch_size': 512,
    'lr': 0.0001,
    'model': {
        'n_layers': 5,
        'hidden_dim': 512,
    },

    'gan_loss_weight': 1.,
    'dis_lr': 0.0001,
    'dis_beta1': 0.5,
    'discriminator': {
```

```
        'n_layers': 4,
        'hidden_dim': 512,
    },
})

print (regression_with_discriminator_hparams)

#create_and_train_regression_model_with_discriminator(regression_with_discriminator_hparams, data_train, test_data=data_test)
```

```
{'epochs': 20, 'batch_size': 512, 'lr': 0.0001, 'model': {'n_layers': 5, 'hidden_dim': 512}, 'gan_loss_weight': 1.0, 'dis_lr': 0.006
```

```
create_and_train_regression_model_with_discriminator(regression_with_discriminator_hparams, data_train, test_data=data_test)
```

Finally, it is valuable for us to know, how long did it take you to finish this practical?